

Software Development Processes



Dr. Petru Florin Mihancea

Presentation designed and written by human based on:

I. Sommerville - Software Engineering 8, Ch. 4 Software Processes, Ch. 17. Rapid Software Development
R. Pressman - Software Engineering, Ch. 2 The Process
M. Fowler - UML Distilled, Ch2. Development Process
P. Deemer & G. Benefield - SCRUM Primer

V20260303

Software (Development) Process

A software process is a set of activities and associated results that produce a software product

Fundamental software production activities

1. **Software specification**
defining the software to be produced
2. **Software development**
designing, programming/coding, testing (+debugging)
3. **Software validation**
testing to ensure that it is what the customer requires
4. **Software evolution/maintenance**
Software specification, development and, validation for modifications to adapt to customer/market requirements

A software process model = a simplified representation of a process

Jr. Petru Florin Mihancea

Process Models (& Categories)

1. **Waterfall**

separate steps/phases for specification, design, etc.

2. **Evolutionary/Iterative**

interleaves specification, development, validation activities, etc.

fast development of an initial system and refine it with the customer

...

Jr. Petru Florin Mihancea

Comparison

Waterfall

vs.

Evolutionary / Iterative

***breaks down the
project based on activity***

e.g.,
2-months analysis phase
followed by
4-months design phase
followed by
3-months coding phase
followed by
3-months testing phase

***usually, breaks down the
project by subsets of
functionality***

e.g., 3 iterations
1st iteration takes 1/4 of
requirements and perform
analysis, design, code, test;
when iteration ends, we have a
system doing 1/4 of all needed
functionality
2nd iteration takes 1/4 of
requirements and etc.

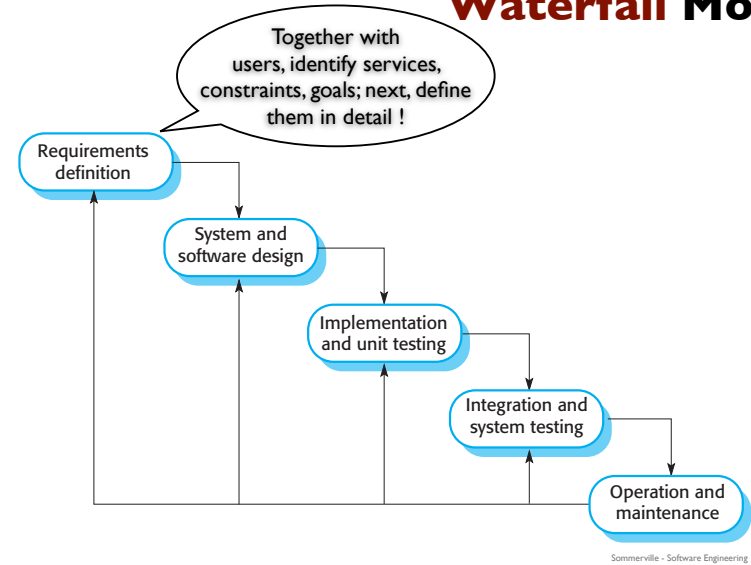
Jr. Petru Florin Mihancea

1

The Waterfall Model

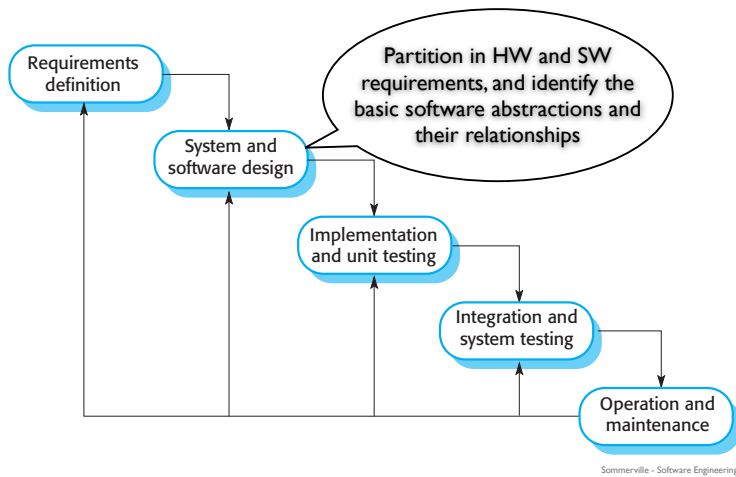
Jr. Petru Florin Mihances

Waterfall Model



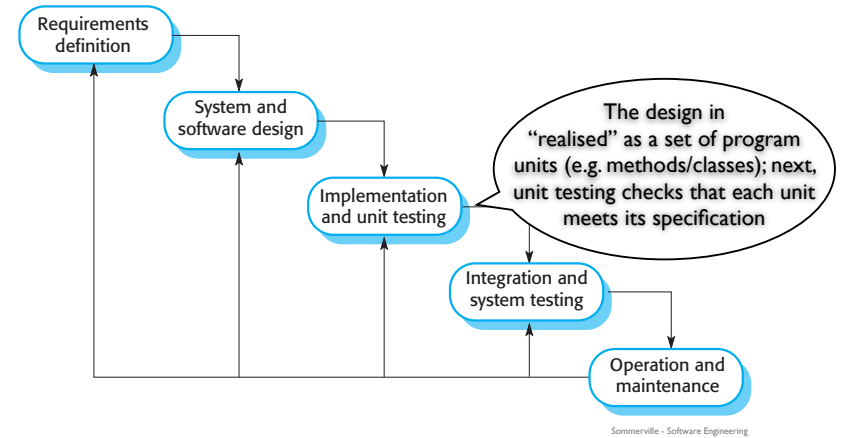
Jr. Petru Florin Mihances

Waterfall Model



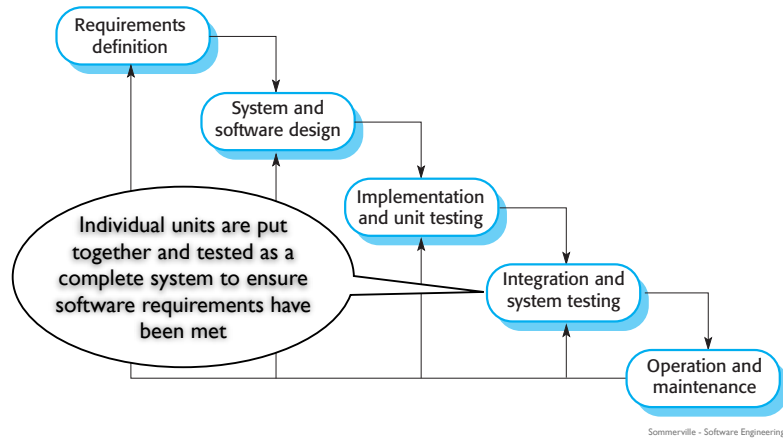
Jr. Petru Florin Mihances

Waterfall Model



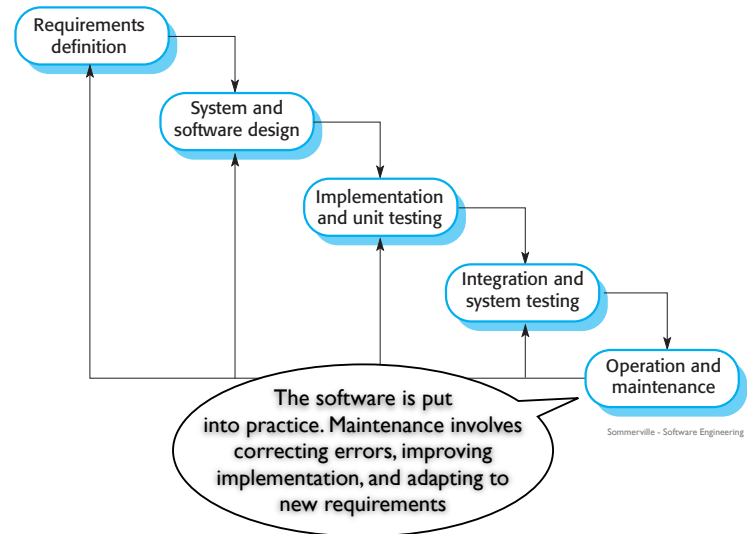
Jr. Petru Florin Mihances

Waterfall Model



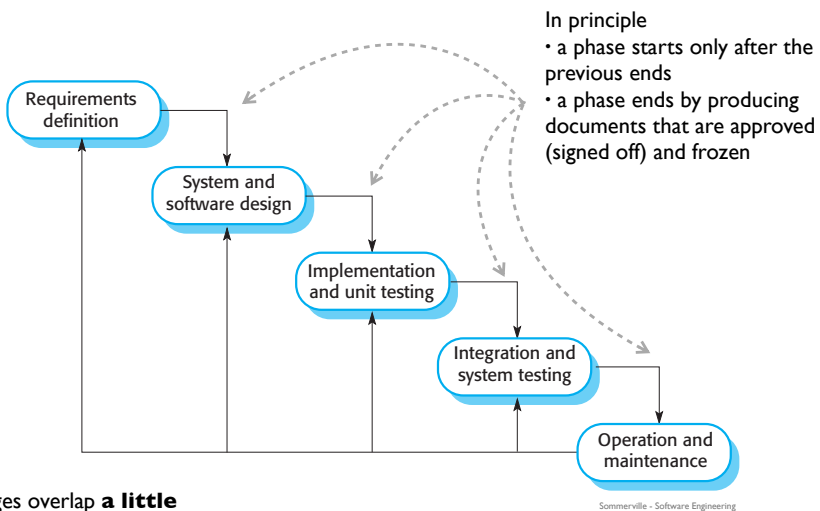
Jr. Petru Florin Mihances

Waterfall Model



Jr. Petru Florin Mihances

Waterfall Model



In practice

- these stages overlap **a little** and feed information to each other but still, after a short time, the previous phase is **frozen**

Jr. Petru Florin Mihances

Pros/Cons

Pros

Documentation is produced at each phase
with implications on various activities (e.g., writing contracts)
The process is more visible (from a managerial perspective)

Cons

inflexible partitioning of the project into distinct stages
premature “freezing” is dangerous

- it is not easy for a customer to explicitly state all the requirements; the software won't do what the user wants
- design flaws will be eliminated via programming hacks

commitments (frozen decisions) made at an early stage make very difficult to respond to changing customer requirements

customer must have patience - a working version comes late in the project time-span

Jr. Petru Florin Mihances

When?

when requirements are **well understood** and **unlikely to radically change during development**

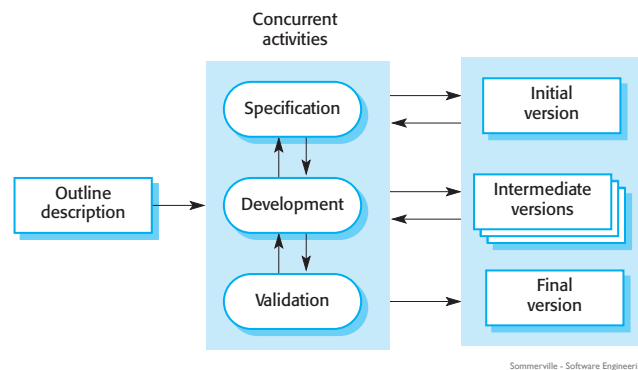
Jr. Petru Florin Mihances

2

Evolutionary/Iterative Models

Jr. Petru Florin Mihances

Basic Idea



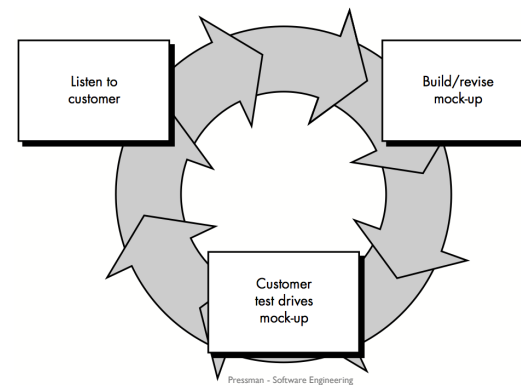
... develop an initial implementation, exposing this to the user and refine it through many versions until an adequate system has been developed

Jr. Petru Florin Mihances

A

Throwaway Prototyping

The **objective is to understand the customer's requirements and hence develop a better requirements definition**



- developer and customer meet and define the overall objectives of software, identify whatever requirements are known
- a quick design occur focusing those aspects visible to the user
- a prototype is built that is then evaluated by the customer to refine requirements, enabling developer to better understand the requirements

Jr. Petru Florin Mihances

Pros/Cons

Both user and developer like the approach

users get quickly a feel of the actual system

developers get to build something immediately

But ...

the users see a “working” version that is held together “with chewing gum and baling wire”

e.g., the developer will make implementation compromises, will neglect long-term maintainability, etc. **only to get fast a working prototype**

DO NOT transform the prototype into a product by applying “a few fixes” (as the customer will beg ;))

Jr. Petru Florin Mihances

Pros/Cons

Both user and developer like the approach

users get quickly a feel of the actual system

developers get to build something immediately

But ...

the users see a “working” version that is held together “with chewing gum and baling wire”

e.g., the developer will make implementation compromises, will neglect long-term maintainability, etc. **only to get fast a working prototype**

DO NOT transform the prototype into a product by applying “a few fixes” (as the customer will beg ;))

... clearly state that **the prototype is built to serve as a mechanism for defining requirements ... it is then discarded** and another paradigm will be used to build the actual software

Jr. Petru Florin Mihances

3

Exploratory Development

The **objective** of this type of processes is to work with the customer to **explore** their requirements and **deliver** a system

start with the parts that are well understood

the system evolves by adding new features proposed by the customer

Pros

specification can be developed incrementally

producing systems that meet immediate needs

Cons

the process is not so visible (from a managerial perspective)

higher risk of poor structure (because of continuous changes)

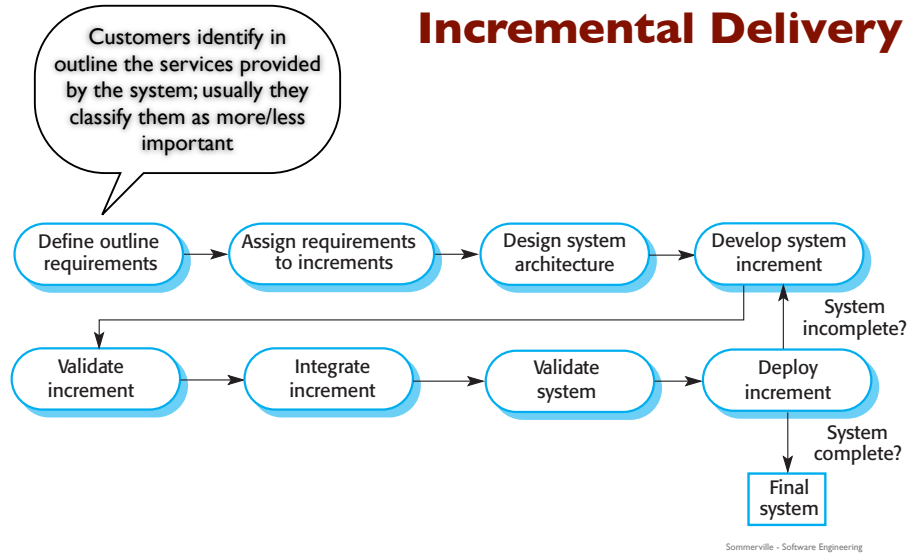
There are different models variations of this kind

Jr. Petru Florin Mihances

Incremental Delivery

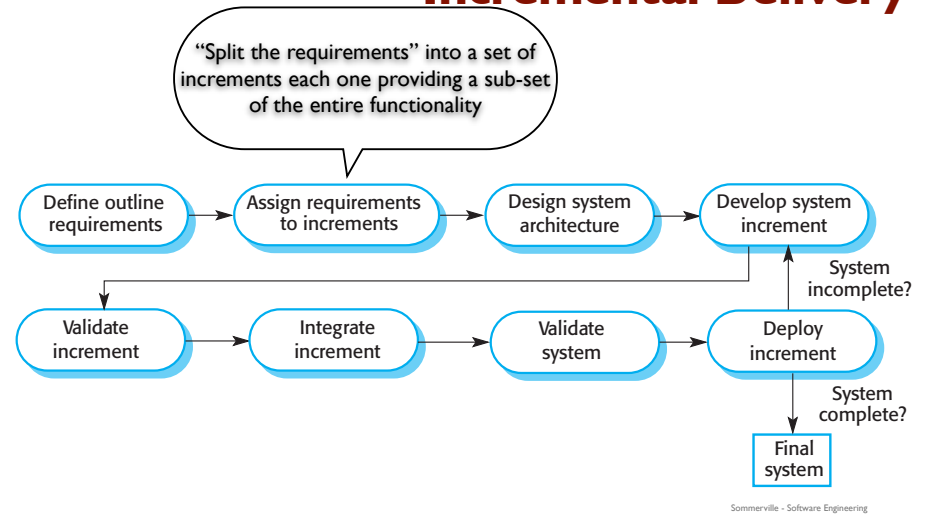
Jr. Petru Florin Mihances

Incremental Delivery



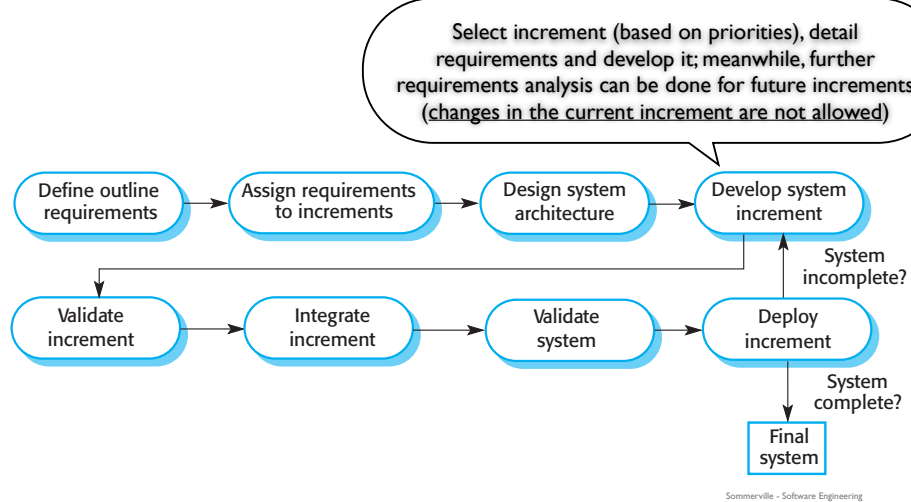
Jr. Petru Florin Mihances

Incremental Delivery



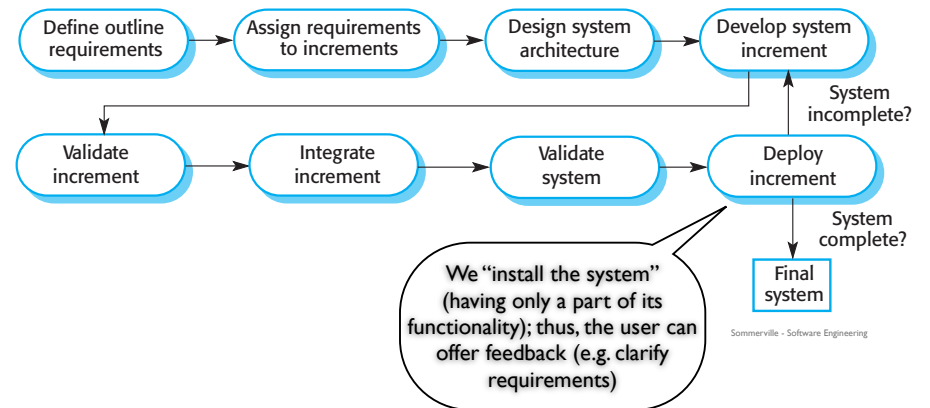
Jr. Petru Florin Mihances

Incremental Delivery



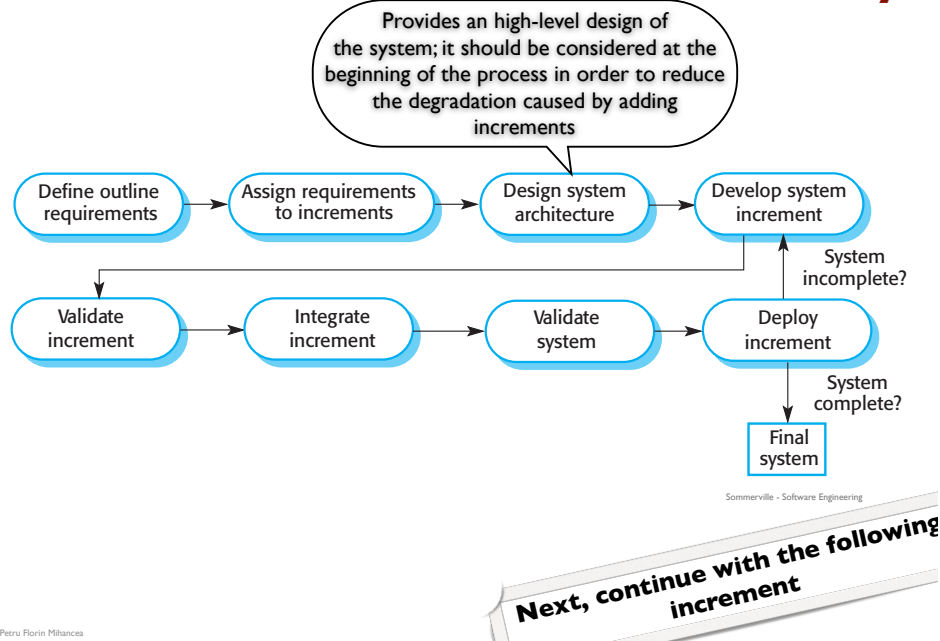
Jr. Petru Florin Mihances

Incremental Delivery



Jr. Petru Florin Mihances

Incremental Delivery



Pros

Can cope with requirements changes

Customers do not have to wait

the increments from the beginning of the process can contain the most important functionality and it can be used by the users

The customer is directly involved

due to user feedback:

more likely to meet the desired requirements

new requirements can be discovered / **better refinement**

Since increments with higher priority are delivered first

the most important functionality receives the most testing

Jr. Petru Florin Mihances

Cons

Management problems

they like processes that generate regular deliverables to assess progress; it may not be time effective to write documents for very fast iterations

Contractual problems

usually based on specifications; it may be difficult to design a contract where requirements are not fully defined from the beginning

Validation problems

due to minimising documentation and iterative specification, how do we validate the system?

Maintenance

continual change tends to degrade the structure of the system; thus special actions **must be considered during development** to avoid degradation

Jr. Petru Florin Mihances

Difficulties

Rework issues

due to modifications, redesign / deletion (in later iterations) of some code might be seen as a waste; but it is better than having poorly designed / patched code

A set of basic functionalities are **common** for different parts of the system; it may be difficult to identify them since requirements are not described in detail

Increments should be small and should deliver some functionality

it can be difficult to map requirements onto an increment of the right size

Jr. Petru Florin Mihances

Increment Size **Solution**

Timeboxing

iterations must be of a fixed length of time :)

if you can't build all you intended to build in an iteration

... slip some functionality from the iteration and

DO NOT SLIP THE DATE of the iteration

Jr. Petru Florin Mihances

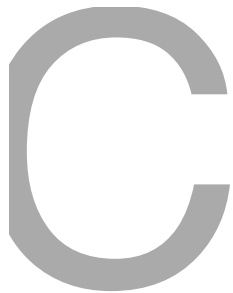
When **NOT** Appropriate?

extremely large systems where teams are working in **different locations**

embedded systems where **software depends on hardware development**

extremely critical systems where all requirements must be analysed to be able to check for interactions that may compromise safety, security and other critical issues

Jr. Petru Florin Mihances

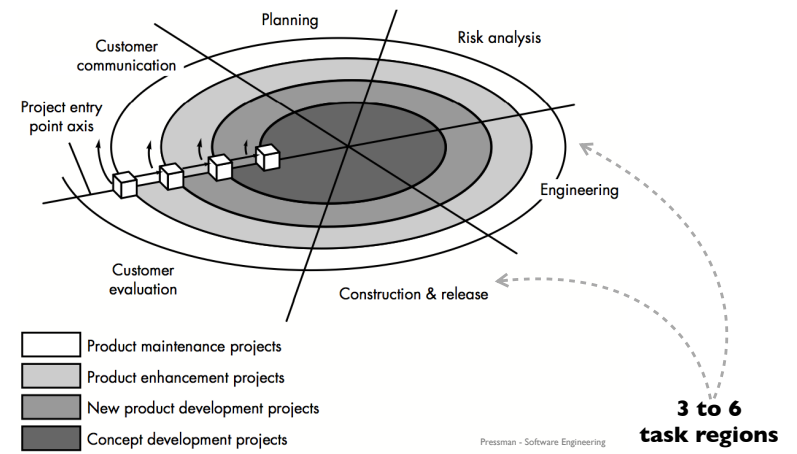


Spiral Model

Combines the development of incremental "versions" with the controlled and systematic aspects of the waterfall model.

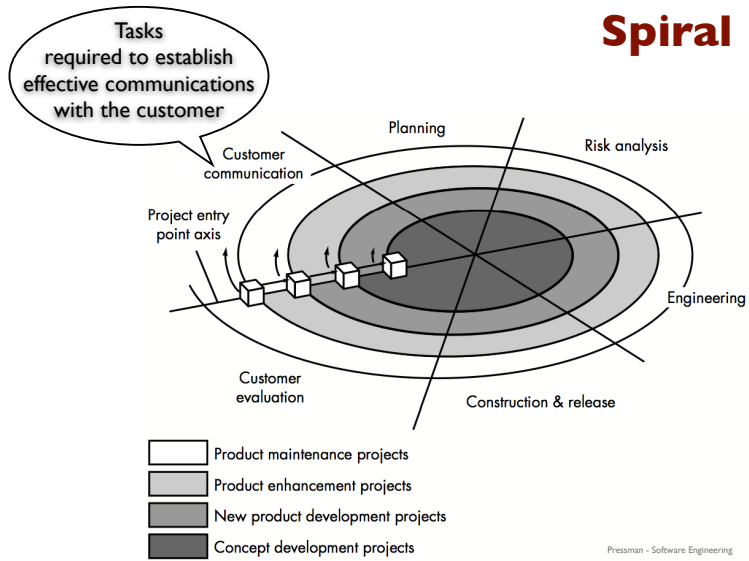
Jr. Petru Florin Mihances

Spiral Model



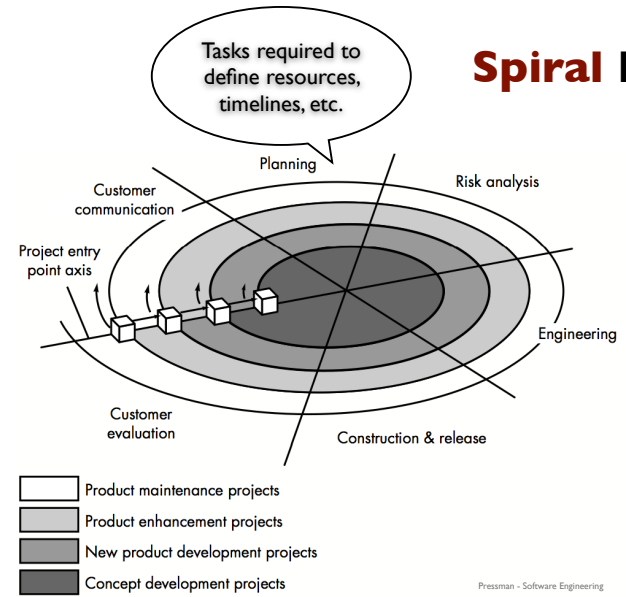
Jr. Petru Florin Mihances

Spiral Model



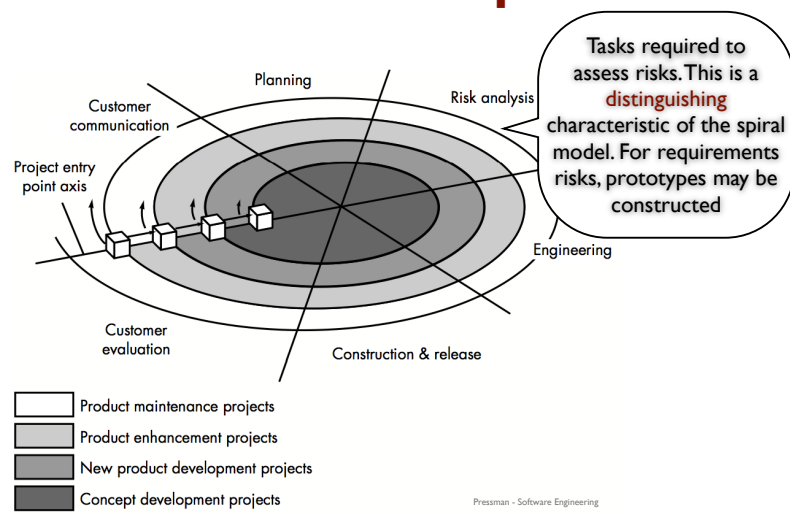
Dr. Petru Florin Mihances

Spiral Model



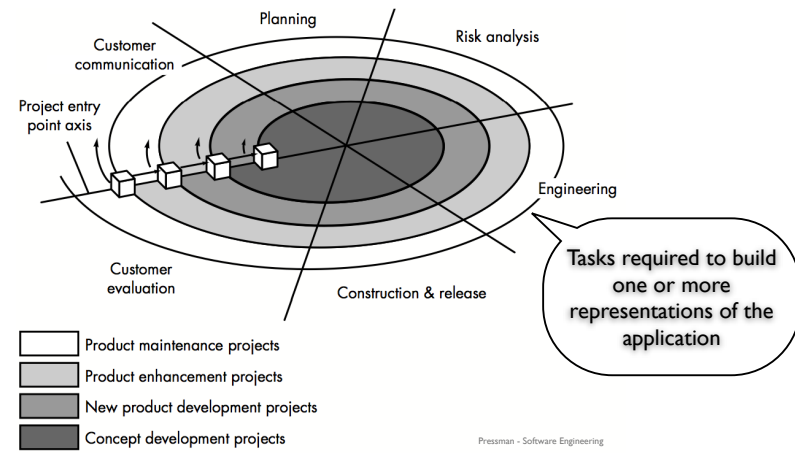
Dr. Petru Florin Mihances

Spiral Model



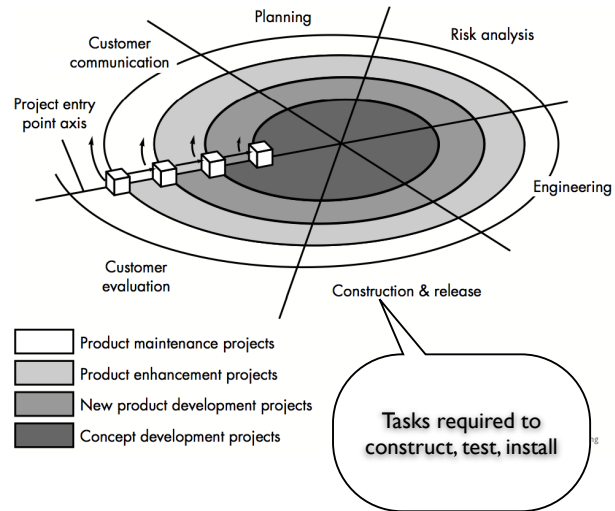
Dr. Petru Florin Mihances

Spiral Model



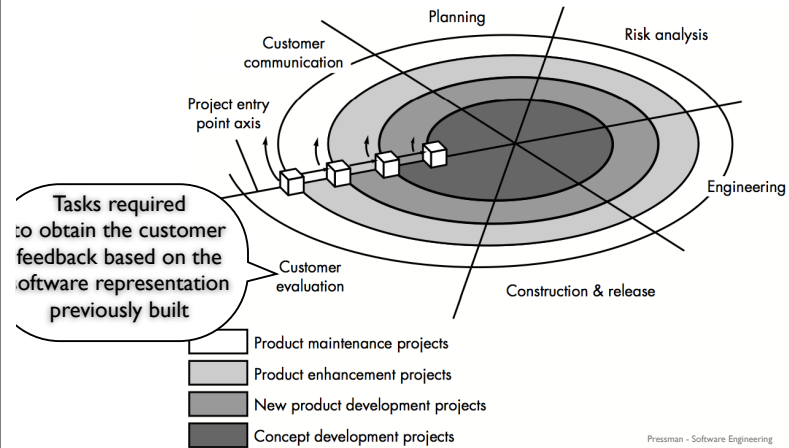
Dr. Petru Florin Mihances

Spiral Model



Dr. Petru Florin Mihances

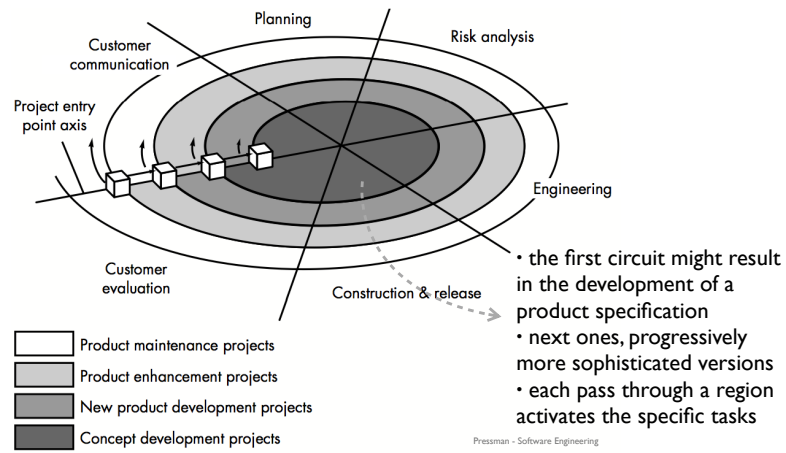
Spiral Model



Pressman - Software Engineering

Dr. Petru Florin Mihances

Spiral Model - View 1

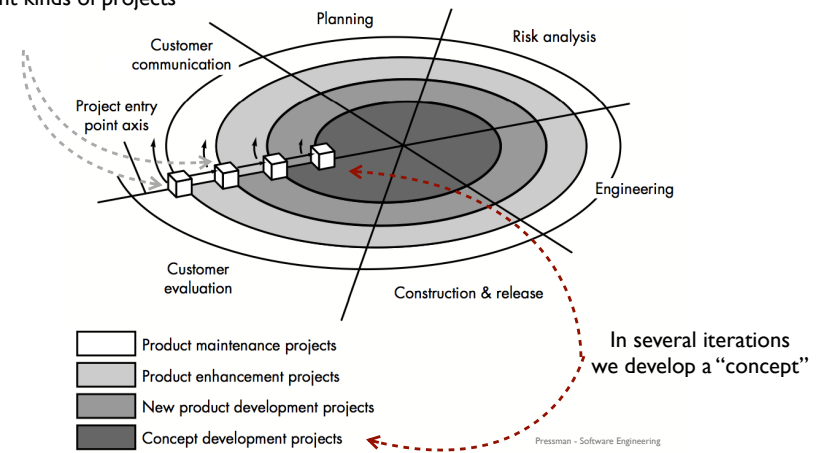


Pressman - Software Engineering

Dr. Petru Florin Mihances

Spiral Model - View 2

Each cube can be used to represent the starting point for different kinds of projects



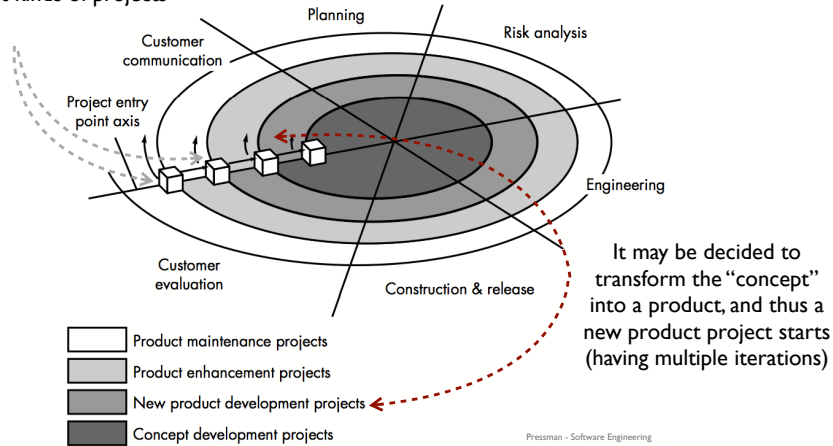
Pressman - Software Engineering

Dr. Petru Florin Mihances

Can be adapted to apply through the entire life of software

Spiral Model - View 2

Each cube can be used to represent the starting point for different kinds of projects



And so on, e.g. where a change is initiated, the process starts at the appropriate entry point e.g. product enhancement

Can be adapted to apply through the entire life of software

Dr. Petru Florin Mihances

Pros/Cons

Pros

explicitly consider risks and uses prototyping as a risk reduction mechanism (at any stage in the evolution of the product)
maintains the systematic stepwise approach suggested by the waterfall life-cycle, but integrates it into an iterative framework

Cons

the model has not been as widely used as other approaches

Dr. Petru Florin Mihances

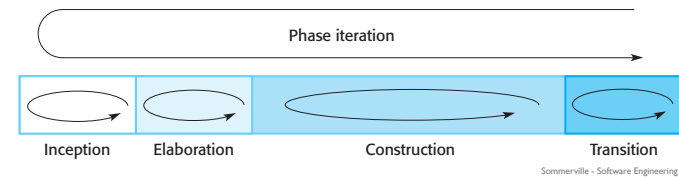
D

Rational Unified Process

Dr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time

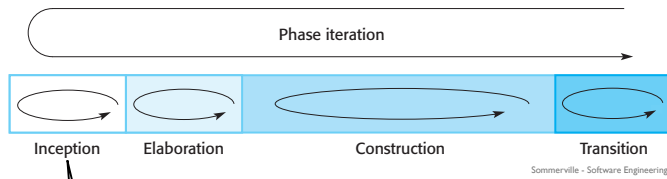


Somehow, reminds us about Waterfall but ... phases are defined by project milestones and goals (often business-oriented)

Dr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time

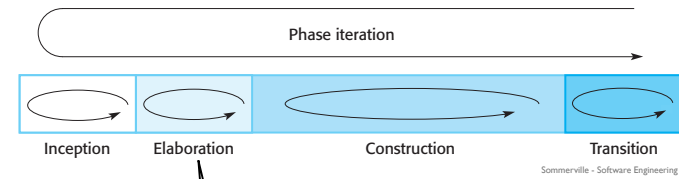


Establish business use cases for the system. Identify external entities (people, other systems) that interact with the system. Next, assess the contribution of the system to the business (should we continue?)

Jr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time

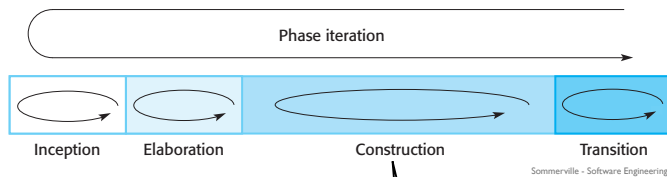


Understand the problem domain, establish an architectural framework, and the project plan, identify risks. At the end we must have: requirements model, architectural description, development plan

Jr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time

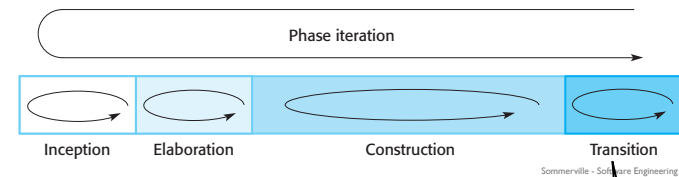


System design, programming and testing. Parts of the system are developed in parallel and integrated during this phase. At the end, we must have the working system + documentation.

Jr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time

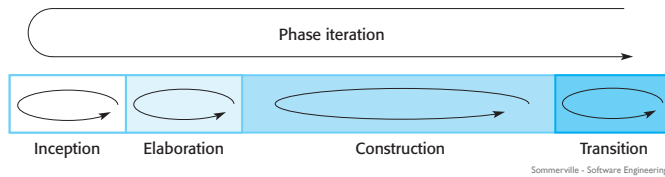


“Move” the system from the developer community to the user community and make it work in the real environment (this is something omitted in other models). At the end: document and working system in the actual working environment

Jr. Petru Florin Mihances

Dynamic Perspective

The phases of the process model over time



Sommerville - Software Engineering

Two ways for iterating:
 a. the result of each phase can be built incrementally
 b. all the phases can be enacted incrementally

Jr. Petru Florin Mihances

Static Perspective (I)

The activities (named workflows)

Workflow	Description
Business modelling	The business processes are modelled using business use cases
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.

Sommerville - Software Engineering

Jr. Petru Florin Mihances

Static Perspective (2)

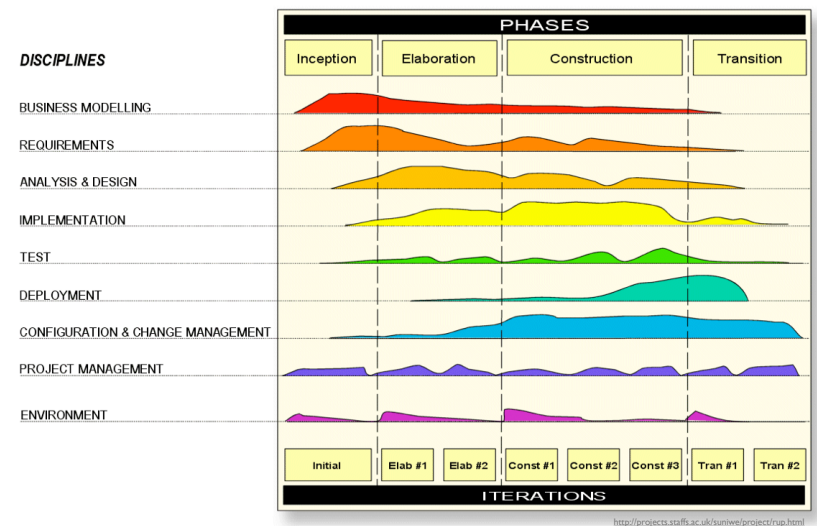
The activities (named workflows)

Workflow	Description
Testing	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users and installed in their workplace.
Configuration and change management	This supporting workflow managed changes to the system (see Chapter 25).
Project management	This supporting workflow manages the system development (see Chapters 22 and 23).
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

Sommerville - Software Engineering

Jr. Petru Florin Mihances

A Combined Perspective



<http://projects.staffs.ac.uk/sumlwe/project/rup.html>

Jr. Petru Florin Mihances

Practice Perspective

Good practices to be used during RUP

Develop software iteratively

plan increments based on customer priorities and
deliver highest priority increments first

Manage requirements

explicitly document user requirements (we will see use cases)
and keep track of changes

Visually model software

UML e.g. class diagrams, sequence diagrams

Jr. Petru Florin Mihancea

Innovations

separation of phases and workflows

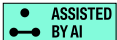
phases are dynamic and have goals (often business oriented)
workflows are static and are technical activities that are not
associated with a single phase

recognition that deploying software in the
user environment is part of the process

Jr. Petru Florin Mihancea

Middle of Chapter II

Live Quiz



Jr. Petru Florin Mihancea

3

Agile Methods

Jr. Petru Florin Mihancea

The Problem

80's - early 90's vision

the best way to achieve better software is through careful planning, formalised quality assurance, the use of analysis methods of CASE tools and **controlled and rigorous** software development processes

Much time is spent on **how** the system should be developed

... justified when developing **very large and/or critical systems**

... but at least questionable if this overhead is greater than the time required to actually build the program (likely to happen in the case of **small-medium business software**)

Jr. Petru Florin Mihances

Solution

Agile methods developed in the 90's

allow teams to **focus on the software itself** rather
on contracts, documents, etc.

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

<http://agilemanifesto.org/>

Jr. Petru Florin Mihances

Principles

1. People not processes

Team members should be left to develop their own ways of working without prescriptive processes

2. Incremental delivery

Software is delivered (rapidly) in increments

3. Customer involvement

Closely involved in the development, providing new and prioritising requirements and evaluate each iteration

4. Embrace change

Expect the requirements to change (even in case of rapid changes during development or very late changes)

5. Maintain simplicity

Focus on simplicity in software and process; work to eliminate complexity from the system

Jr. Petru Florin Mihances

Difficulties

The **need of a customer who is willing** and able to spend time with the development team

Prioritising changes
can be difficult

Maintaining simplicity
requires extra work

Contractual problems
the customer pays for the time required for development and not for a specific set of requirements

Jr. Petru Florin Mihances

When?

best suited for **small to medium sized** software **without critical** aspects that may affect security and safety

Jr. Petru Florin Mihances



Extreme Programming (XP)

Jr. Petru Florin Mihances

Basics

Pushes good practices (e.g. iterative delivery) and customer involvement to “extreme”

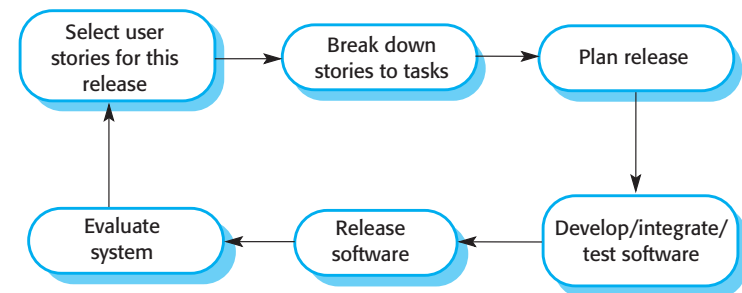
new versions may be built several times a day

all tests must be run for every new version and must be successful; only in that case the version is accepted

increments are delivered to customer roughly every 2 weeks

Jr. Petru Florin Mihances

XP Release Cycle



Sommerville - Software Engineering

Jr. Petru Florin Mihances

Practices (A)

I. On-Site Customer

A representative of the end user should be available full time for the use of XP team (thus, it is actually a member of the team) responsible for defining **acceptance tests**, involved in **specifying and in prioritising requirements**

in the form of **story cards / user stories / scenarios**

But what is a **Story Card** ?

Jr. Petru Florin Mihances

Story Card

The form in which requirements are captured; it encapsulates a customer's need from the system (usually in terms of an interaction scenario with the system)

Downloading and printing an article

First, you select the article that you want from a displayed list. You then have to tell the system how you will pay for it—this can either be through a subscription, through a company account or by credit card.

After this, you get a copyright form from the system to fill in. When you have submitted this, the article you want is downloaded onto your computer.

You then choose a printer and a copy of the article is printed. You tell the system printing has been successful.

If the article is a print-only article, you can't keep the PDF version, so it is automatically deleted from your computer.

Sommerville - Software Engineering

Jr. Petru Florin Mihances

Tasks

Task 1: Implement principal work flow

Task 2: Implement article catalog and selection

Task 3: Implement payment collection

Payment may be made in 3 different ways. The user selects which way they wish to pay. If the user has a library subscription, then they can input the subscriber key which should be checked by the system. Alternatively, they can input an organizational account number. If this is valid, a debit of the cost of the article is posted to this account. Finally, they may input a 16 digit credit card number and expiry date. This should be checked for validity and, if valid a debit is posted to that credit card account.

Sommerville - Software Engineering

Jr. Petru Florin Mihances

Practices (B)

2. Small & frequent releases

The minimum set of functionalities providing useful value from the system is developed first; following releases are frequent and add functionality to the previous release

3. Incremental planning

The stories developed in a **particular release** are determined / selected based on the time available and story priorities

As requirements change, unimplemented story cards may change or may be discarded; more over, new cards may appear and re prioritisation may occur

Jr. Petru Florin Mihances

Practices (B)

4. Simple design

Enough design is performed in order to meet the current requirements and no more - e.g via UML in sketch mode

5. Refactoring

Frequent code change tends to degrade the internal structure of the application

Consequently, all developer are expected to refactor/re-organise the code asap when the possibility of improvement is found; thus the code should remain simple and maintainable

6. Automated tests & Test-first development

An **automated** unit test framework is used to write tests for a new piece of functionality **before** that functionality is implemented

Jr. Petru Florin Mihances

Testing in XP

A. Automated testing mechanisms

Usually, a test will be an executable component simulating the submission of the inputs to the tested entity and checks that the output meets the entity specification

Whenever a new functionality is added, the tests can be run quickly and problems introduced by the new code can be caught immediately

B. When and based on what?

a. Test first-development

Writing tests first **defines the interface** and **the specification of the behaviour** that is going to be implemented

Can help clarify requirements i.e. in order to write a test you must clearly understand the specification

Avoids the tendency to skip testing in order to maintain the schedule

Jr. Petru Florin Mihances

Testing in XP (2)

b. Incremental test development from scenarios

Stories are split into **tasks** each of which represents a single feature of the system; tests can be derived then for each task

c. User involvement in test development and validation

The user must help to produce the **acceptance tests** i.e. tests with user data to check that the application meets the user stories

Difficulties

Many programmers do not like writing tests

Some tests might be extremely difficult to write before the system is implemented

The user commitment to the tests development (how do we know that her tests are sufficiently complete ?)

Jr. Petru Florin Mihances

Practices (C)

7. Sustainable pace

Large amounts of overtime are not acceptable

8. Collective ownership

One may work in all areas of the system; all the developers own all the code; anyone can change anything; no islands of expertise develop

9. Pair Programming

Very simple, programmers work in pairs; they actually **sit together** at the same workstation :)

Jr. Petru Florin Mihances

Advantages of Pair Programming

Supports the idea of **common code ownership**

The code is owned by the team as a whole

Individuals are not responsible for problems, **the entire team is responsible**

It acts as an **informal review process**

Each line of code is looked at by at least two people

Not as good as a formal review process, but cheaper and faster

Support for **refactoring/restructuring**

Restructuring **is a long term investment and takes time**; thus, an **individual practicing refactoring might be seen as less efficient**

Due to pair programming and common code, everybody gains from refactoring and it is more likely to support the process

Jr. Petru Florin Mihances

Isn't this a waste of resources?

NO!

There is some evidence that **productivity of pair programming is comparable with that of two people working independently**

The pair members discuss various solutions so they might have fewer "false starts" and thus, less rework

Errors might be avoided because of the informal review of the code

Jr. Petru Florin Mihances

Practices (D)

10. **Continuous Integration**

As soon as the work on a task is completed it is **integrated** into the whole system; after any such integration, **all the unit tests must pass**

Keeps the team in sync avoiding painful integration cycles

To achieve this:

The entire build of the application (i.e. compiling, copying resource files into place, etc.) should be an automatic process (and ideally, should be fast)

Automated testing mechanism are required

The developers must be announced immediately when some tests fail (e.g. via email)

There are dedicated CASE tools to support continuous integration

Jr. Petru Florin Mihances

3

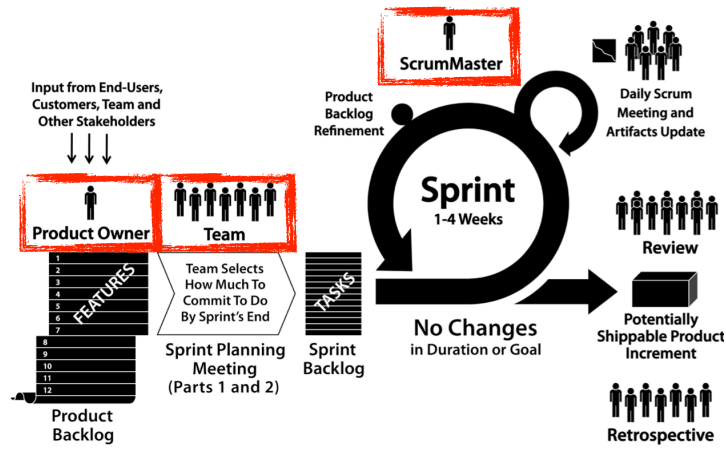
SCRUM



<http://en.wikipedia.org/wiki/File:Scrum.jpg>

Jr. Petru Florin Mihances

SCRUM



P. Deemer, G. Benefield - Scrum Primer Version 1.2

Iterative & incremental agile process
the cycles are named **sprints** and are **timeboxed**

Jr. Petru Florin Mihances

Product Owner

One person

Identifies product features, **prioritise** them, **decides** which of them must be in the **top of the list for the next sprint**, and continuously **re-prioritise and refine the list**

Prioritisation must be made to maximise the “profit” (e.g. revenues, highest-value-lowest-cost features, satisfy important clients, etc.)

Sometimes may be directly the customer (for internal applications)

The Team

7 (+/- 2) people

Builds the product indicated by the **Product Owner**

100% dedicated to one product during a sprint (avoid multiple-project involvement)

Cross-Functional - includes all the necessary expertise (analysis, development, testing, database design, etc.)

Multi-skilled members (but they not have to be all generalists)

Self-Organising - the team decides what to commit to in a sprint and how to accomplish that commitment

Jr. Petru Florin Mihances

Roles

Roles (2)

ScrumMaster

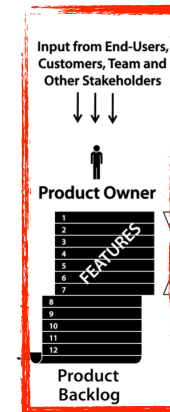
He is **not** the manager of the team (e.g., does not tell people what to do, does not assign tasks, etc.)

Helps the group learn, apply and reinforce Scrum rules (e.g, pushback the Product Owner if he tries to add new deliverables in the middle of a sprint)

May be an active member of the Team (in the case of small teams) but cannot be the Product Owner

Jr. Petru Florin Mihances

Entering Scrum



Product Owner constructs and continuously updates a prioritised list of items called the Product Backlog

Item

- (new) customer features
- engineering improvements goals (e.g., solve a scalability issue)
- known defects

Items are usually expressed using **use cases** or **user stories**

The subset of items intended to be solved for a particular release of the product forms the **Release Backlog**

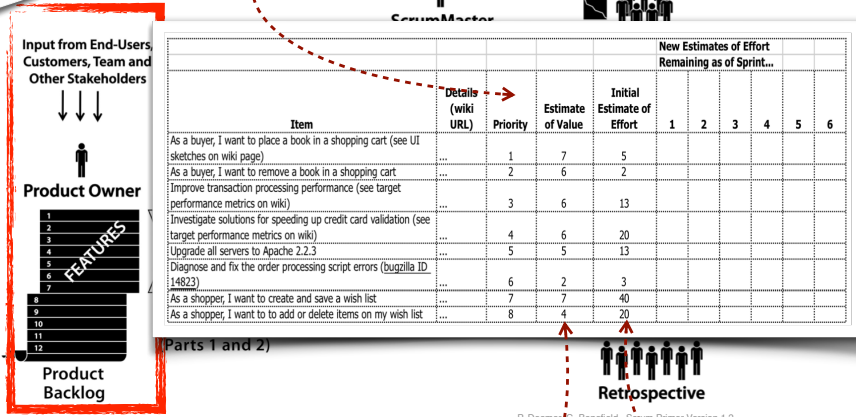
P. Deemer, G. Benefield - Scrum Primer Version 1.2

Jr. Petru Florin Mihances

Entering Scrum

No recipe for effort estimation estimate in terms of relative size using a "point" as unit
After several sprints the Team can find how much can do in a sprint (e.g., 26 points/sprint)
When stable this is the team **velocity**

Product Owner sets priorities
e.g., highest-value-lowest-cost



P. Deemer, G. Benefield - Scrum Primer Version 1.2

© Petru Florin Mihances

Sprint Planning Meeting

Part I

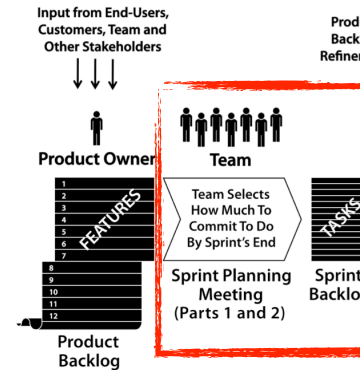
Product Owner + Team

- review **highest-priority items** in Product Backlog (e.g., goal, context, etc.)

Part 2

Team (Product Owner is not required)

- selects the items that **the team commits to complete by the end of the sprint** (starting from the top of the Product Backlog)
- the Team may lobby for a particular low-priority item when it fits easily with a high-priority one
- based on estimating the available working hours (e.g., usually 4-6 h / day / member for sprint related-work, but also must consider vacations, etc.)
- based on discussing/designing/etc. the top items, the Team breaks them into tasks for each of which it estimates the required time to complete



© Petru Florin Mihances

The Sprint Backlog

Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	New Estimates of Effort Remaining as of Day...					
				1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database		5						
	create webpage (UI)		8						
	create webpage (Javascript logic)		13						
	write automated acceptance tests		13						
	update buyer help webpage		3						
Improve transaction processing performance	merge DCP code and complete layer-level tests		5						
	complete machine order for pRank		8						
	change DCP and reader to use pRank http API		13						

P. Deemer, G. Benefield - Scrum Primer Version 1.2

NO item is added to the Sprint Backlog during the sprint

Positive influence of the Team feeling protected, knowing exactly its objectives

Product Owner is really thinking on the item prioritization and knows what the Team committed to do

Product Owner can work on updating the remaining of the Product Backlog

People volunteer for tasks one at a time, when it is time to pick a new one

The task status should be marked: Not started, In Progress, Completed

© Petru Florin Mihances

Sprint - Daily Scrum

15 minutes daily meeting - each member is standing and reports:

- 1) What I get done from the last meeting ?
- 2) What I try to finish by the next meeting ?
- 3) Found impediments

• opportunity for synchronisation, self-coordination and obstacle report

• ScrumMaster must react to blocking situations to try to solve them (e.g., maybe during a follow-up meeting)

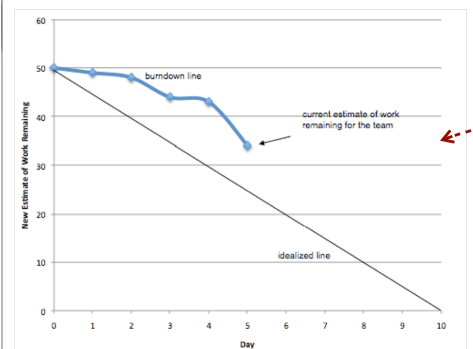


P. Deemer, G. Benefield - Scrum Primer Version 1.2

© Petru Florin Mihances

Sprint - Updating Sprint Backlog

Product Backlog Item	Sprint Task	Volunteer	Initial Estimate of Effort	New Estimates of Effort Remaining at end of Day...					
				1	2	3	4	5	6
As a buyer, I want to place a book in a shopping cart	modify database	Sanjay	5	4	3	0	0	0	0
	create webpage (UI)	Jing	3	3	3	2	0	0	0
	create webpage (Javascript logic)	Tracy & Sam	2	2	2	2	1	0	0
	write automated acceptance tests	Sarah	5	5	5	5	5	0	0
	update buyer help webpage	Sanjay & Jing	3	3	3	3	3	0	0
Improve transaction processing performance	merge DCP code and complete layer-level tests	...	5	5	5	5	5	5	5
	complete machine order for pRank	...	3	3	8	8	8	8	8
	change DCP and reader to use pRank http API	...	5	5	5	5	5	5	5
	...	...	...	...	...	...	...	...	...
Total for person			50	49	48	44	43	34	



Each day, members estimate the amount of time until the task is done

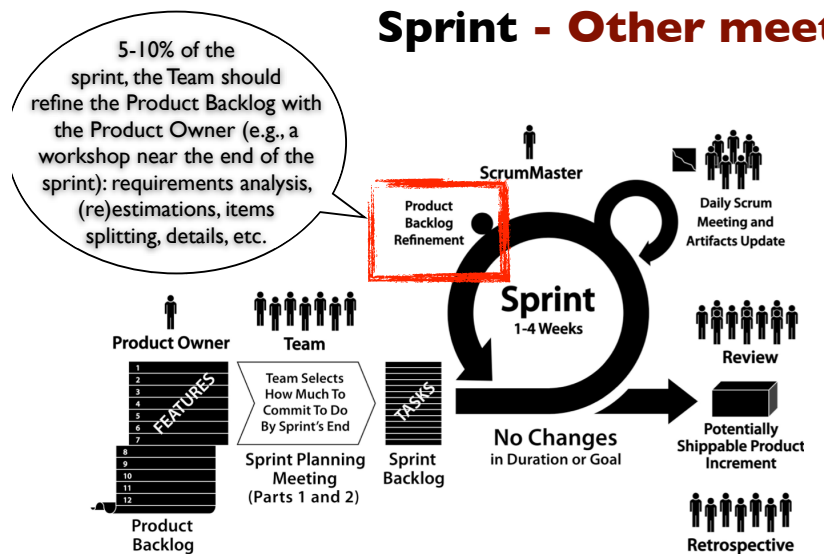
Sprint Burndown Chart

The Team can easily observe if the goal of the sprint can be met in the remaining time or adjustments are required (e.g., slip functionality)

Jr. Petru Florin Mihances

P. Deemer, G. Benefield - Scrum Primer Version 1.2

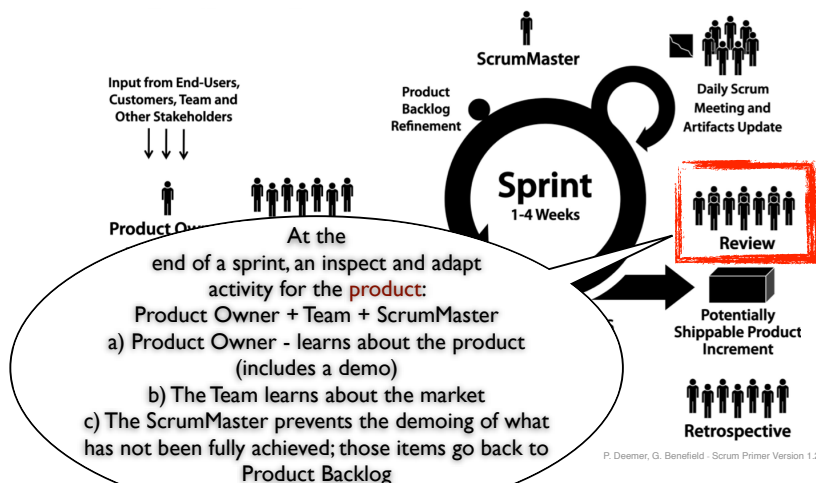
Sprint - Other meetings



P. Deemer, G. Benefield - Scrum Primer Version 1.2

Jr. Petru Florin Mihances

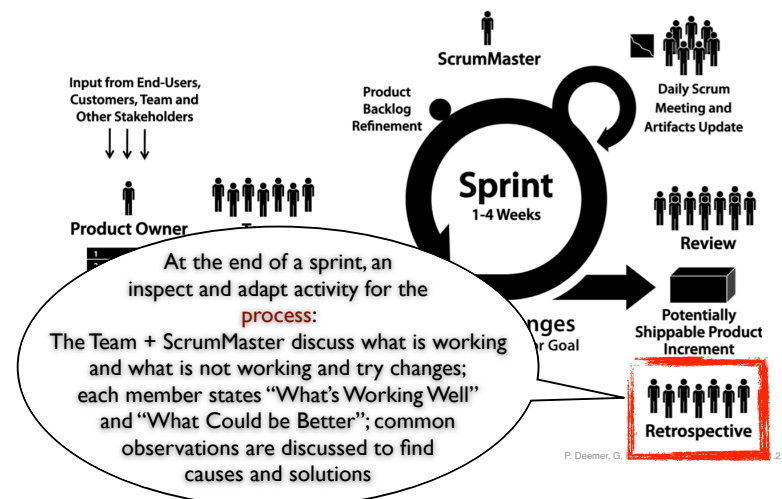
Sprint - Other meetings



P. Deemer, G. Benefield - Scrum Primer Version 1.2

Jr. Petru Florin Mihances

Sprint - Other meetings



P. Deemer, G. Benefield - Scrum Primer Version 1.2

Jr. Petru Florin Mihances

Other elements

The product must be **shippable** at the end of each sprint

In practice

- An actual release will contain more items (i.e., **Release Backlog Items**) developed in several sprints
- The Release Backlog must be maintained in a similar manner like the Sprint Backlog, together with a **Release Burndown Chart**
- A final **release sprint** might be required before a release
 - Perfect vision of shippable product after each sprint is hard to achieve

End of Chapter II

Live Quiz

